MECHANIQ SETUP & RUN GUIDE

Complete Step-by-Step Instructions for Running the App after Pulling on GitHub

Frontend: React Native / Expo SDK 57 (Expo Router)

Backends: Node.js (3002, 5002) + Python FastAPI (8000)

Target OS: Android (arm64-v8a)

🚀 QUICKSTART: WHAT TO DO AFTER INSTALLING EVERYTHING

Once you have pulled the repository and installed dependencies, follow these **exact 5 steps** in your terminals to run the app on your phone:

1 Connect Phone via USB

Plug your Android phone into your computer. Ensure **Developer Options** and **USB Debugging** are enabled.

Run `adb devices` to verify.

2 Start the Backend Microservices (Terminal 1)

In the project root folder (`m/`), launch all 3 backend APIs with one command:

```
node scripts/start-backends.js
```

→ Leave this terminal running! Wait until you see: `[READY] ALL THREE BACKEND SERVICES ARE OPERATIONAL`.

3 Forward the Ports to your Phone (Terminal 2)

Open a second terminal window and route backend ports over USB so the phone can reach your computer:

```
adb reverse tcp:3002 tcp:3002; adb reverse tcp:5002 tcp:5002; adb reverse tcp:8000 tcp:8000; adb reverse tcp:8081 tcp:8081
```

4 Start the App with NPM (Terminal 2)

In the second terminal, start the Expo application:

```
npm start
```

When the Expo menu appears in your terminal, press **'a'** on your keyboard to launch the app on your Android device!

5 Log in as Customer or Mechanic

- **Customer:** Log in with customer email → opens Customer Dashboard.
- **Mechanic:** Tap *Create Account* → *I'm a Mechanic* → system generates username (e.g. `mike@mechanic`) → log in with username & password → opens Mechanic Dashboard!

1 System Architecture Overview

MechanIQ uses three local microservices alongside Firebase Cloud for authentication and data storage:

Service	Tech Stack	Port	Purpose
Mobile App	React Native 0.86, Expo 57	`Metro: 8081`	Main mobile app with dual Customer and Mechanic dashboards.
Picture Search API	Node.js / Express / Gemini	`Port 3002`	AI image recognition to search & identify car parts from photos.
OBD-II Diagnostic API	Node.js / Express	`Port 5002`	Live vehicle PID telemetry, engine sensors & fault codes.
Sound Detection API	Python / FastAPI / TensorFlow	`Port 8000`	CNN engine audio anomaly detection from recorded engine sounds.
Auth & Database	Firebase Auth & RTDB	`Cloud`	Real-time chat, appointment bookings, role verification.

2 Prerequisites Checklist

Before running the app, ensure these tools are installed on your workstation:

- **Node.js:** Version `20.x` or `22.x` (LTS recommended).
- **Python:** Version `3.10` or `3.11` (for Sound Detection AI backend).
- **Java JDK:** JDK 17 (e.g. Eclipse Adoptium Temurin 17 or Android Studio JBR). *(Do not use Java 24/25 to avoid C++ Prefab restriction errors).*
- **Android Studio & SDK:**
  - Android SDK Platform 36 and Build-Tools 36.0.0.
  - NDK version `27.1.12297006`.
  - `adb` added to system `PATH` (check by running `adb version`).
- **Physical Android Phone:** Connected via USB with **USB Debugging** enabled.

3 Initial Setup: Clone & Install Dependencies

Run these commands once when first setting up the project:

```
# 1. Clone your GitHub repository
git clone <YOUR_GITHUB_REPO_URL>
cd m

# 2. Install frontend npm dependencies
npm install
```

> **Zero Native Build Hassle**
> Shims for `@react-native-community/netinfo`, `expo-crypto`, `expo-notifications`, and `expo-secure-store` are bundled in `src/shims/` and mapped in `metro.config.js`. No manual native linking is required.

4 Environment Configuration (Files to Create)

Create these two configuration files in the project root directory (`m/`):

File 1: `.env` (Default Endpoints)

```
EXPO_PUBLIC_PICTURE_SEARCH_API_URL=http://127.0.0.1:3002
EXPO_PUBLIC_OBD_API_URL=http://127.0.0.1:5002
EXPO_PUBLIC_SOUND_DIAGNOSIS_API_URL=http://127.0.0.1:8000
```

File 2: `.env.local` (Secret Keys & Python Path)

```
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
GEMINI_MODEL=gemini-3.8-flash
SERPAPI_API_KEY=<YOUR_SERPAPI_KEY>
MECHANIQ_SOUND_PYTHON=python
EXPO_PUBLIC_PICTURE_SEARCH_API_URL=http://127.0.0.1:3002
EXPO_PUBLIC_OBD_API_URL=http://127.0.0.1:5002
EXPO_PUBLIC_SOUND_DIAGNOSIS_API_URL=http://127.0.0.1:8000
```

5 Python Backend Setup (Sound Detection)

Set up the Python virtual environment for the AI audio detection service:

```
# 1. Navigate to sound-detection directory
cd backend/sound-detection

# 2. Create and activate virtual environment
python -m venv venv
.\venv\Scripts\activate       # On Windows PowerShell
# source venv/bin/activate    # On macOS / Linux

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Return to root
cd ../..
```

6 Detailed Instructions: Running the App Every Day

Terminal 1: Start the Microservices

Run the master startup script from the root folder. It initializes Picture Search, OBD-II, and Sound Detection simultaneously:

```
node scripts/start-backends.js
```

Confirm that all three services display `[READY]`:

- Picture Search → `http://127.0.0.1:3002/health`
- OBD API → `http://127.0.0.1:5002/health/ready`
- Sound Detection → `http://127.0.0.1:8000/health/ready`

Terminal 2: Forward Ports & Launch Expo

```
# 1. Connect ports over USB
adb reverse tcp:3002 tcp:3002
adb reverse tcp:5002 tcp:5002
adb reverse tcp:8000 tcp:8000
adb reverse tcp:8081 tcp:8081

# 2. Start the app
npm start
```

Press `a` in Terminal 2. Expo will install and launch the MechanIQ development build directly onto your phone.

Alternative: Building & Installing the Release APK

If you want to install a standalone, production-ready APK on your phone without running Metro:

```
# Set JDK 17
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"

# Build APK with Gradle
cd android
.\gradlew.bat :app:assembleRelease --no-daemon

# Install onto your phone
adb install -r app/build/outputs/apk/release/app-release.apk
```

7 User Roles & Login Credentials Guide

MechanIQ uses strict role-based routing. Below is how you test both experiences:

Role	How to Register	How to Log In	Destination Dashboard
Customer	Tap "*Create Account*" → "*I'm a Customer*". Enter email & password.	Enter your email (e.g. `john@gmail.com`) and password.	**Customer Dashboard** (`/(customer)`) Diagnostics, Fuel, Marketplace, Profile.
Mechanic	1. Tap "*Create Account*" → "*I'm a Mechanic*". 2. Enter name (e.g. *Mike*). System generates: `mike@mechanic`. 3. Provide valid communication email, workshop info, and create password. 4. Submit → dialog confirms: **Your Username: mike@mechanic**.	Enter your generated username (e.g. `mike@mechanic`) and password.	**Mechanic Dashboard** (`/(mechanic)`) Jobs, Live Diagnostics, Customer Roster, Profile.

Role Exclusivity Enforced

The app uses `RoleRouteGuard`. Customers are blocked from mechanic screens and automatically redirected to `/(customer)`. Mechanics are blocked from customer screens and automatically redirected to `/(mechanic)`.

8 Common Troubleshooting Scenarios

"Fetch failed" or "Cannot reach sound detection backend"

- Make sure `node scripts/start-backends.js` is running in Terminal 1.
- Run `adb reverse tcp:3002 tcp:3002; adb reverse tcp:5002 tcp:5002; adb reverse tcp:8000 tcp:8000` in Terminal 2.
- Ensure your phone is connected with USB Debugging enabled.

"Java restricted method called / Gradle build fails"

Ensure Gradle uses **JDK 17**. Set `$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"` (or your Adoptium 17 path) before compiling.

"Windows PowerShell script execution disabled"

Run `npm.cmd start` or `npx.cmd expo start`, or run: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.